



Публичный API

testing_lab.refunds

Документ описывает публичные функции и классы из `testing_lab/refunds.py`. Бизнес-ограничения приведены по `testing_lab/CONTRACT.md`; тесты служат только подтверждением ожидаемого поведения.

calculate_refund

```
def calculate_refund(
    items: list[dict],
    requested: dict[int, int],
) -> Decimal
```

Назначение: рассчитать сумму возврата для выбранных позиций заказа.

Параметры:

- `items` — позиции заказа. У каждого элемента обязателен `product_id`, поскольку функция строит индекс по всему списку. Для позиции, выбранной к возврату, нужны `quantity` и `unit_price`; `discount_percent` необязателен и при отсутствии считается равным нулю.
- `requested` — отображение идентификатора товара на количество, запрошенное к возврату.

Результат: `Decimal`, приведённый к точности до двух знаков после запятой. Сумма рассчитывается по цене позиции из заказа; при скидке используется цена после её вычета.

Подтверждённые исключения: `ValueError`, если список выбранных позиций пуст, товар отсутствует в заказе, количество не положительно или превышает купленное.

Ограничения: можно вернуть часть или всё купленное количество позиции, но нельзя запросить больше купленного. В `requested` должна быть хотя бы одна позиция.

Пример:

```
from decimal import Decimal

from testing_lab.refunds import calculate_refund

items = [{"product_id": 101, "quantity": 2, "unit_price": Decimal("25.00")}]
amount = calculate_refund(items, {101: 1}) # Decimal("25.00")
```

ReturnRepository

Хранилище заявок на возврат.

ReturnRepository.__init__

```
def __init__(self, connection: sqlite3.Connection) -> None
```

Назначение: принять соединение SQLite, создать таблицу `returns`, если её ещё нет, и зафиксировать транзакцию.

Параметры: `connection` — открытое соединение `sqlite3.Connection`.

Результат: `None`. Соединение передаётся репозиторию извне.

Подтверждённые исключения: явные исключения в методе не заданы.

Ограничения: этот учебный `ReturnRepository` реализован с использованием `sqlite3`. Это деталь данной реализации, а не заявление о том, что SQLite является общей production-базой приложения.

ReturnRepository.create

```
def create(self, order_id: int, amount: Decimal) -> dict
```

Назначение: сохранить заявку для заказа.

Параметры: `order_id` — идентификатор заказа; `amount` — рассчитанная сумма возврата.

Результат: словарь сохранённой заявки с полями `id`, `order_id`, `amount` (`Decimal`) и `status`. Начальный статус — `requested`. После вставки изменения фиксируются.

Подтверждённые исключения: явные исключения в методе не заданы.

Ограничения: метод сохраняет переданную сумму; проверку условий возврата выполняет не репозиторий, а `RefundService`.

ReturnRepository.get

```
def get(self, return_id: int) -> dict | None
```

Назначение: получить сохранённую заявку по идентификатору.

Параметры: `return_id` — идентификатор заявки.

Результат: словарь с полями `id`, `order_id`, `amount` (`Decimal`) и `status`; если записи с таким идентификатором нет — `None`.

Подтверждённые исключения: явные исключения в методе не заданы.

Ограничения: метод только читает заявку и не изменяет её.

RefundService

Сервис проверки условий возврата и создания заявки.

RefundService.__init__

```
def __init__(self, repository: ReturnRepository) -> None
```

Назначение: сохранить переданный репозиторий для создания заявок.

Параметры: `repository` — экземпляр `ReturnRepository`.

Результат: `None`.

Подтверждённые исключения: явные исключения в методе не заданы.

RefundService.create_request

```
def create_request(
    self,
    order: dict,
    requested: dict[int, int],
    today: date,
) -> dict
```

Назначение: проверить возможность возврата, рассчитать сумму и создать заявку через репозиторий.

Параметры:

- `order` — заказ с полями `id`, `status`, `delivered_at` и `items`.
- `requested` — запрошенные к возврату количества по идентификаторам товаров; ограничения те же, что у `calculate_refund`.
- `today` — дата, относительно которой проверяется срок возврата.

Результат: словарь заявки, возвращённый `ReturnRepository.create`.

Подтверждённые исключения: `ValueError`, если статус заказа не `delivered`, дата `delivered_at` отсутствует, она позже `today`, либо с даты получения прошло больше 14 календарных дней. Также передаётся `ValueError`, если выбранные позиции или количества нарушают ограничения `calculate_refund`.

Ограничения: дата получения обязательна; возврат разрешён с даты получения по 14-й календарный день включительно. При успешном создании сумма заявки совпадает с результатом `calculate_refund`, а начальный статус равен `requested`.

Пример:

```
from datetime import date
import sqlite3

from testing_lab.refunds import RefundService, ReturnRepository

connection = sqlite3.connect(":memory:")
repository = ReturnRepository(connection)
service = RefundService(repository)

order = {
    "id": 1001,
    "status": "delivered",
    "delivered_at": date(2026, 9, 1),
    "items": [{"product_id": 101, "quantity": 2, "unit_price": "25.00"}],
}
created = service.create_request(order, {101: 1}, date(2026, 9, 2))
# created содержит сохранённую заявку со статусом "requested".
connection.close()
```

Поведенческие гарантии

- Для расчёта достаточно данных позиций из заказа; применяется сохранённая цена, учитывается необязательная скидка, результат имеет два знака после запятой.
- Все проверки статуса, даты доставки, срока и выбранных количеств выполняются до создания заявки.
- Новая заявка сохраняется через `ReturnRepository` с рассчитанной суммой и статусом `requested`.
- Срок в 14 календарных дней включителен; дата `today` не может предшествовать дате получения.

Границы ответственности

- `calculate_refund` рассчитывает сумму и проверяет выбранные позиции и количества, но не проверяет статус заказа и срок возврата.
- `RefundService` проверяет условия возврата и передаёт сохранение заявки репозиторию.
- `ReturnRepository` отвечает за сохранение и чтение заявок; он не выполняет бизнес-проверки возврата.
- Описанный API — Python API модуля. HTTP endpoints здесь не заявляются.
- `sqlite3` относится к учебной реализации `ReturnRepository` и не объявляется общей production-базой приложения.